RV College of Engineering
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Non-Human Identity Governance Agent

MAJOR PROJECT REPORT
(IS481P)

Submitted by,

Manu Prakash Bhat **1RV22IS035**

Under the guidance of

Dr. Merin Meleet
Associate Professor
Dept. of ISE
RV College of Engineering

Mr. Tejeshwar Rao
Manager DevSecOps & Cloud
DevSecOps & Cloud Engineering
Honeywell Technology Solutions

In partial fulfillment for the award of degree of

Bachelor of Engineering
in
Information Science and Engineering

Department of ISE
2025-26

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Non-Human Identity Governance Agent

A Major Project Report (IS481P)

Submitted by,

Manu Prakash Bhat **1RV22IS035**

Under the guidance of

Dr. Merin Meleet
Associate Professor
Dept. of ISE
RV College of Engineering

Mr. Tejeshwar Rao
Manager DevSecOps & Cloud
DevSecOps & Cloud Engineering
Honeywell Technology Solutions

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Information Science and Engineering

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Information Science and Engineering

CERTIFICATE

Certified that the major project (IS481P) work titled ***Non-Human Identity Governance Agent*** is carried out by **Manu Prakash Bhat** (**1RV22IS035**) who is bonafide student of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	Head of the Department	Principal
Dr. Merin Meleet	Dr. G S Mamatha	Dr. K. N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

I, **Manu Prakash Bhat** student of eighth semester B.E., Department of Information Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Non-Human Identity Governance Agent**' has been carried out by me and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Information Science and Engineering** during the year 2025-26.

Further I declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

I also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and I will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. Manu Prakash Bhat(1RV22IS035)	

ACKNOWLEDGEMENTS

I am indebted to my guide, **Dr. Merin Meleet**, Associate Professor, Department of ISE, RVCE for the wholehearted support, suggestions and invaluable advice throughout my project work and also helped in the preparation of this thesis.

My sincere thanks to the project coordinator **Dr. Vanishree K**, Associate Professor, Department of ISE for the timely instructions and support in coordinating the project.

My gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

My sincere thanks to **Dr. G S Mamatha**, Professor and Head, Department of Information Science and Engineering, RVCE for the support and encouragement.

I express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

I thank all the teaching staff and technical staff of Information Science and Engineering department, RVCE for their help.

Lastly, I take this opportunity to thank my family members and friends who provided all the backup support throughout the project work.

ABSTRACT

The rapid expansion of cloud-native architectures and DevOps automation has driven an exponential increase in Non-Human Identities (NHIs) such as service principals, API keys, deploy tokens, and managed identities, which now outnumber human users by ratios exceeding 45:1 in enterprise environments. The OWASP NHI Top 10 (2025) indicates that 70% of organisations maintain plaintext secrets in repositories, while credential-based attacks consistently rank among primary breach vectors. This security gap motivated the development of an autonomous governance agent for continuously discovering, classifying, and managing NHIs across cloud ecosystems.

The project implemented a four-layer modular architecture using Python 3.11. The discovery layer employed five specialised agents interfacing with Microsoft Graph API, GitHub REST API, Azure Key Vault SDK, and SSL/TLS connections to enumerate credentials across GitHub and Azure environments. A deterministic risk scoring engine quantified each identity on a 100-point composite scale combining type weight, credential age, privilege scope, and dormancy signals, supplemented by Isolation Forest anomaly detection and linear regression trend prediction. Policy enforcement operated through eleven codified rules in Python and OPA Rego. The remediation layer provided automated credential rotation via Ed25519 key generation and PyNaCl sealed-box encryption. A React 19 TypeScript dashboard delivered real-time monitoring across eleven interactive pages.

The system achieved 82% unit test coverage and completed full-scan cycles of 500 repositories in approximately 61 seconds. The policy engine identified violations across all eleven rules, with the ML anomaly detector achieving reliable separation of high-risk credentials. Compliance scoring against SOC 2, ISO 27001, NIST CSF, and CIS Benchmarks provided automated audit-readiness assessment, measurably reducing the attack surface from NHI sprawl.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

API Application Programming Interface

CI/CD Continuous Integration / Continuous Deployment

Ed25519 Edwards-curve Digital Signature Algorithm (Curve 25519)

GA GitHub Actions

IAM Identity and Access Management

ISO International Organization for Standardization

ML Machine Learning

NHI Non-Human Identity

NIST National Institute of Standards and Technology

OPA Open Policy Agent

OWASP Open Worldwide Application Security Project

PAT Personal Access Token

REST Representational State Transfer

SDK Software Development Kit

SOC System and Organisation Controls

SP Service Principal

SSH Secure Shell

SSL Secure Sockets Layer

TLS Transport Layer Security

ZTA Zero Trust Architecture

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

Chapter 1

Introduction

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

CHAPTER 1

INTRODUCTION

The governance of machine credentials represents one of the most pressing yet under-addressed challenges in modern enterprise security. This chapter establishes the context and motivation for the Non-Human Identity Governance Agent, defines the problem space it addresses, states the project objectives and methodology, and outlines the organisation of this report.

1.1 Introduction

The term Non-Human Identity refers to any credential, token, certificate, or service account that enables machine-to-machine communication without direct human involvement. In contemporary cloud-native environments, these identities manifest as Application Programming Interface (API) keys, Personal Access Tokens (PATs), deploy tokens, Service Principals (SPs), managed identities, Secure Shell (SSH) keys, and Transport Layer Security (TLS) certificates. Their proliferation is a natural consequence of microservice architectures, Continuous Integration / Continuous Deployment (CI/CD) pipelines, and infrastructure-as-code practices that characterise modern software delivery.

What makes these identities particularly concerning from a security perspective is the combination of their sheer volume and the relative neglect they receive compared to human user accounts. Enterprise environments routinely maintain tens of thousands of machine credentials, many of which hold elevated privileges, lack rotation schedules, and persist long after the workloads they were created for have been decommissioned. Each unmanaged credential represents a potential entry point for adversaries who have learned that attacking a dormant service principal is far easier than compromising an account protected by multi-factor authentication and behavioural monitoring.

The Open Worldwide Application Security Project (OWASP) Non-Human Identities Top 10 report published in 2025 brought formal recognition to this category of risk, identifying ten critical vulnerability patterns including improper credential offboarding, secret leakage in repositories, and overprivileged service accounts [1]. The report found that seven out of ten organisations maintain plaintext secrets in their source code repositories. This statistic alone illustrates the gap between the security investment directed at human identities and the comparatively minimal governance applied to their machine

counterparts.

1.2 Motivation

Several converging factors motivated the development of an automated governance agent for non-human identities. The first and most significant is the scale of the problem. Industry analyses have established that Non-Human Identitys (NHIs) outnumber human identities at ratios ranging from 17:1 in moderately sized organisations to over 45:1 in large enterprises with extensive automation [2], [3]. Managing this volume of credentials manually is simply not feasible, yet the majority of organisations lack any automated tooling for NHI lifecycle management.

The second motivating factor is the severity of the threat. Credential-based attacks consistently account for a dominant proportion of data breaches. The Verizon Data Breach Investigations Report has identified credential abuse as the primary attack vector in nearly half of all recorded incidents [4]. When adversaries obtain a long-lived, unrotated service principal with broad permissions, they gain persistent access that is difficult to detect through conventional monitoring because the credential usage appears legitimate.

A third consideration is the regulatory landscape. Compliance frameworks including System and Organisation Controls (SOC) 2, International Organization for Standardization (ISO) 27001, and National Institute of Standards and Technology (NIST) Cybersecurity Framework all mandate controls for credential lifecycle management, access reviews, and least-privilege enforcement [5], [6], [7]. Organisations that cannot demonstrate automated governance of their machine credentials face audit findings and potential regulatory consequences. The absence of a unified tool that combines discovery, scoring, policy enforcement, and remediation across cloud platforms creates both a security vulnerability and a compliance gap.

During my internship at Honeywell Technology Solutions, I observed firsthand how large engineering teams struggle with credential sprawl across GitHub repositories and Azure subscriptions. Service principals created for proof-of-concept projects remained active months after the projects concluded. Deploy keys granted write access persisted unchanged across multiple team rotations. These observations confirmed that the problem is not theoretical but a daily operational reality in enterprise environments.

1.3 Problem Statement

Enterprise organisations deploying workloads across GitHub and Azure cloud ecosystems lack a unified, autonomous system for continuously discovering non-human identities, quantifying their risk posture, enforcing governance policies, and executing automated remediation. Existing tools address individual aspects of the credential lifecycle in isolation, requiring manual coordination between discovery, assessment, and rotation processes. This fragmented approach results in unmanaged credentials persisting beyond their useful life, accumulating excessive privileges, and escaping the periodic access reviews that human accounts receive. The project addresses this gap by implementing an integrated governance agent that operates across the full NHI lifecycle from discovery through remediation.

1.4 Objectives

The primary objective of this project is to design and implement an autonomous agent that continuously discovers non-human identities across GitHub repositories and Azure cloud services, encompassing personal access tokens, deploy keys, Actions secrets, service principals, managed identities, Key Vault secrets, and TLS certificates. The second objective focuses on quantifying the risk associated with each discovered identity through a deterministic 100-point scoring algorithm that combines credential type, age, privilege scope, and dormancy indicators, augmented by machine learning anomaly detection using Isolation Forest and trend prediction via linear regression. The third objective is to enforce codified governance policies through eleven rules implemented in both Python and Open Policy Agent (OPA) Rego, enabling distributed policy evaluation with transparent fallback mechanisms. The fourth objective addresses automated credential rotation for deploy keys and Actions secrets, employing Edwards-curve Digital Signature Algorithm (Curve 25519) (Ed25519) key generation and sealed-box encryption to execute remediation without human intervention. The fifth and final objective is to deliver a real-time monitoring dashboard built with React and TypeScript that provides security operators with interactive visibility into the entire NHI inventory, risk trends, policy violations, and compliance posture across multiple regulatory frameworks.

1.5 Brief Methodology of the Project

The project followed an iterative development methodology structured across three phases. The first phase established the discovery and monitoring foundation, implementing Secure Sockets Layer (SSL)/TLS certificate checking, GitHub NHI enumeration, and Azure service principal scanning. The second phase introduced the intelligence layer with risk scoring, policy enforcement via OPA, and the machine learning analysis engine. The third phase delivered the remediation capabilities and the interactive React dashboard for operational use.

Each phase followed a cycle of requirements analysis, component implementation, integration testing, and refinement based on real-world scanning results against the target enterprise environment. The architecture was designed as a set of loosely coupled Python agents coordinated by a central orchestrator, enabling independent development and testing of each discovery module before integration into the full pipeline. The choice of Python 3.11 provided mature async capabilities and extensive library support for the required cloud Software Development Kit (SDK) integrations, while React 19 with TypeScript offered type safety and component-based architecture for the frontend monitoring interface.

Figure 1.1: Three-phase development methodology

The methodology diagram in Figure 1.1 illustrates the progression from foundational discovery through intelligence and finally to operational remediation and monitoring capabilities.

1.6 Assumptions made / Constraints of the Project

The system operates under several assumptions about the deployment environment. It assumes the availability of a GitHub PAT with sufficient read scopes to enumerate repositories, deploy keys, and Actions secrets for the target organisation. Azure scanning requires service principal credentials with Graph API read access for service principal enumeration and Key Vault reader permissions for secret and certificate discovery. Net-

work connectivity to the target cloud environments must be available from the execution host, and the Python 3.11 runtime with all dependencies specified in the requirements file must be installed.

The primary constraints of the project relate to the boundaries of automated remediation. PAT rotation cannot be performed programmatically through the GitHub API and therefore requires manual operator action guided by the system recommendations. The dashboard operates without built-in authentication, assuming deployment behind a corporate reverse proxy or within a trusted network boundary. The Machine Learning (ML) analysis requires a minimum of three historical scan snapshots before trend prediction becomes meaningful, which means the first several runs of the system produce limited predictive output. Finally, the OPA binary is an optional dependency; when unavailable, the system transparently falls back to the equivalent Python policy engine without loss of enforcement coverage.

1.7 Organization of the Report

The remainder of this report is organised into five chapters followed by appendices. Chapter 2 presents a thematic literature survey covering the NHI threat landscape, zero trust frameworks, AI-driven anomaly detection, policy-as-code governance, and existing commercial tools, culminating in an identification of the research gap that this project addresses. Chapter 3 details the system architecture and component design across the four-layer modular structure, including the dashboard architecture, security design principles, and data flow specifications. Chapter 4 describes the implementation of each layer, covering the development environment, agent implementations, orchestrator logic, and React frontend construction. Chapter 5 presents the testing methodology and results, including unit test coverage, performance benchmarks, security analysis findings, machine learning evaluation metrics, and a comparative assessment against existing tools. Chapter 6 concludes with a summary of achievements, outlines future scope for extending the system, and reflects on the learning outcomes gained through the project.

Chapter Summary

This chapter established the context of non-human identity governance, articulated the problem of credential sprawl in enterprise cloud environments, and defined the objectives and methodology of the project. The following chapter surveys the academic

literature and industry landscape that informed the system design.

Chapter 2

Literature Survey

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

CHAPTER 2

LITERATURE SURVEY

This chapter surveys existing research, industry reports, and commercial tools relevant to non-human identity governance, AI-driven security automation, and policy-as-code enforcement. The review identifies the theoretical foundations underpinning the proposed NHI Governance Agent and highlights persistent gaps in current approaches that motivate this work.

2.1 Introduction to the Survey

The literature review spans three domains: academic research on identity governance, anomaly detection, and zero-trust architectures; industry reports and surveys quantifying the scale and severity of the NHI threat landscape; and analysis of existing commercial tools that address parts of the NHI governance problem. Rather than treating each source in isolation, this survey synthesises findings thematically to build a coherent picture of the current state of the art and the opportunities for innovation.

2.2 NHI Threat Landscape and Scale of the Problem

The proliferation of cloud-native architectures, microservice deployments, and DevOps automation pipelines has produced a massive expansion in the number of non-human identities operating within enterprise environments. Enterprise-wide analyses have revealed that NHIs now outnumber human identities at ratios ranging from 10:1 to as high as 144:1, with year-over-year growth rates approaching 44% [3]. This exponential growth is driven by the increasing reliance on service principals, API keys, deploy tokens, managed identities, and CI/CD pipeline secrets that enable machine-to-machine communication at cloud scale.

The security implications of this growth are severe. Credential-based attacks consistently rank among the most prevalent breach vectors globally, with credential abuse identified as the primary attack path in nearly half of all recorded data breaches [4]. The OWASP Non-Human Identities Top 10 report identifies the most critical risks facing organisations, including improper offboarding of machine credentials, secret leakage in source code repositories, and overprivileged service accounts [1]. The same report found that 70% of organisations have plaintext secrets stored in code repositories and 47% maintain unused Identity and Access Management (IAM) roles that remain active and

exploitable.

Surveys of security practitioners further underscore the severity of the challenge. A survey of over 800 IT and security professionals found that only 15% of organisations report high confidence in their ability to prevent NHI-related attacks, while 69% express significant concern about the state of their machine identity security posture [8]. The primary pain points identified include inadequate auditing capabilities, insufficient privilege management controls, and the absence of automated policy enforcement for non-human credentials. Analyst reports on the NHI management market confirm that the machine-to-human identity ratio exceeds 17:1 even in moderately sized organisations and project significant market growth as enterprises recognise NHI governance as a board-level security priority [2], [9]. Credential access tactics documented in established threat intelligence frameworks demonstrate that adversaries actively target long-lived, unrotated machine credentials as a reliable means of establishing persistent access to compromised environments [10].

2.3 Zero Trust and Identity Access Management Frameworks

The zero trust security model has emerged as the dominant paradigm for securing modern cloud environments, replacing the traditional perimeter-based approach with the principle that no entity—human or machine—should be implicitly trusted. The foundational architecture defines three core tenets: verify explicitly, enforce least privilege, and assume breach [7]. While originally formulated for human users, these principles apply with equal urgency to non-human identities that operate autonomously and often hold elevated privileges.

Research on IAM in cloud environments has explored challenges posed by dynamic resource provisioning and ephemeral workloads. Studies examining IAM with a focus on Zero Trust Architecture (ZTA) identify the tension between automated credential provisioning and the requirement for continuous verification [11]. Cross-cloud federated IAM systems using trust score computation and behavioural modelling have been proposed to enforce least-privilege access across cloud boundaries [12]. Surveys of IAM in multi-cloud environments document the fragmented state of identity governance practices [13].

The emergence of autonomous AI agents introduces additional governance challenges.

Research on zero-trust identity frameworks for agentic AI demonstrates the inadequacy of conventional authentication protocols for autonomous agents [14]. Complementary work on decentralised identity management integrates self-sovereign identity principles and zero-knowledge proofs into multi-domain orchestration frameworks [15]. Digital identity guidelines and international standards for information security management provide the compliance frameworks against which NHI governance solutions must be evaluated [5], [6], [16].

2.4 AI-Driven Trust Assessment and Anomaly Detection

The application of machine learning to security operations has gained significant traction as organisations seek to move beyond static, rule-based monitoring. Comprehensive surveys of anomaly detection methodologies categorise approaches into statistical, classification-based, clustering-based, and information-theoretic methods [17]. The isolation forest algorithm, which detects anomalies by measuring random partitions required to isolate data points, has proven effective for identifying outlier behaviour in high-dimensional datasets such as NHI access patterns [18].

Research on AI-driven dynamic trust assessment proposes frameworks using both supervised and unsupervised ML models to generate adaptive trust scores for identity verification [19]. Rather than binary authenticated-or-not decisions, these systems compute continuous trust values reflecting the evolving risk profile of each identity. This approach directly informs risk-scoring engine design for NHI governance. The broader landscape of ML in security operations documents both the promise and challenges of deploying detection systems in production, including scarcity of labelled data and the cost of false positives [20]. The emergence of large language model-based autonomous agents introduces new possibilities for security orchestration with agentic systems capable of reasoning about complex scenarios [21].

2.5 Policy-as-Code and Governance Automation

The policy-as-code paradigm represents a shift from manual, document-based governance to machine-executable policy definitions that can be version-controlled, tested, and automatically enforced. Systematic reviews identify OPA with its Rego policy language as the leading framework for expressing governance constraints in declarative, auditable

form [22], [23]. This enables organisations to codify rules such as maximum credential age and permission scope boundaries as testable code integrating into CI/CD pipelines.

Systematic literature reviews on software secret management document the pervasive challenge of secret sprawl—the uncontrolled proliferation of credentials across repositories and configuration files [24]. Studies on automated credential lifecycle management demonstrate that programmatic rotation can reduce exposure windows from months to hours [25]. Standards bodies establish rotation schedules and key management practices informing NHI governance policy rules [26].

The integration of security into DevOps workflows has been surveyed through practitioner studies. Reports on DevSecOps adoption reveal that automated security testing achieves faster remediation times [27]. Performance research demonstrates that elite-performing organisations invest in automated governance controls reducing manual overhead while strengthening security posture [28]. Security challenges specific to microservice architectures, where inter-service credentials grow quadratically with service count, have been systematically analysed [29].

2.6 Existing Tools and Their Limitations

Several commercial and open-source tools address aspects of the NHI governance problem, though none provides the comprehensive, autonomous governance capability required by modern enterprise environments.

HashiCorp Vault is an industry-standard secrets management platform that provides dynamic secret generation, encryption as a service, and fine-grained access policies based on identity [30]. It excels as a centralised secrets store and supports programmatic credential rotation through its API. However, Vault does not autonomously discover NHIs across multi-platform environments, does not perform AI-driven risk scoring of credential postures, and does not operate as an agentic governance system capable of independently identifying and remediating policy violations without human intervention.

Microsoft Entra Workload ID provides identity services for workloads running in Azure, supporting managed identities and federated credentials for cloud-native applications [31]. It offers basic lifecycle policies such as credential expiry notifications for Azure-native service principals. However, its scope is limited to the Azure ecosystem and it lacks cross-platform NHI correlation with GitHub tokens, CI/CD pipeline secrets, or third-party service credentials. It provides no AI-driven risk classification or anomaly

detection capabilities.

Both tools, along with other solutions reviewed, share a common limitation: they address individual components of the NHI governance lifecycle—discovery, storage, rotation, or monitoring—but none integrates all four functions into a continuously operating autonomous agent. The certificate lifecycle management challenges documented in industry research further highlight the operational gaps that emerge when expiry monitoring, policy enforcement, and automated renewal are handled by disconnected tools [32].

2.7 Research Gap and Rationale

The survey reveals a persistent and well-documented gap in the current landscape of NHI security. Academic research has established the theoretical foundations for zero-trust identity frameworks, AI-driven anomaly detection, and policy-as-code enforcement. Industry reports have quantified the scale and severity of the NHI threat, demonstrating that unmanaged machine credentials represent one of the most significant attack surfaces in modern enterprise environments. Yet no existing solution—whether academic prototype or commercial product—combines autonomous cross-platform NHI discovery, AI-driven continuous risk scoring, policy-as-code enforcement with machine-executable governance rules, and automated credential rotation into a single unified governance agent.

The NHI Governance Agent implemented in this project was designed to close this gap. By integrating LangChain-based agentic AI orchestration for intelligent risk assessment, OPA Rego policies for codified governance enforcement, and automated rotation via API-driven workflows, the system brought together capabilities that had previously existed only in isolation. The agent operates continuously and autonomously, eliminating the manual intervention bottlenecks and inter-tool coordination overhead that characterise current approaches.

2.8 Feasibility Study

The technical feasibility of this project was confirmed through the robust availability and maturity of the required cloud platform APIs, AI orchestration frameworks, and infrastructure automation tools. The Azure SDK for Python and the Microsoft Graph API provided comprehensive programmatic access to Azure Active Directory service principals, application registrations, and managed identities [33], [34]. The GitHub Repre-

sentational State Transfer (REST) API enabled enumeration of deploy keys, Actions secrets, and repository-level NHIs [35]. LangChain provided the agentic orchestration framework used for AI-driven risk scoring and decision-making [36], while OPA with Rego supported the declarative policy-as-code enforcement layer [23]. Azure Key Vault offered secure credential storage and rotation capabilities [37], and GitHub Actions (GA) provided the CI/CD automation platform for executing governance workflows on configurable schedules [38]. Python 3.11, selected as the primary implementation language, offered mature async capabilities and extensive library support for the required integrations [39]. The React 19 framework with TypeScript provided the frontend monitoring dashboard, while scikit-learn supplied the machine learning models for anomaly detection and trend prediction. All technologies were validated as production-ready through their use in the implemented system.

Chapter Summary

This chapter surveyed the academic research, industry reports, and commercial tools relevant to NHI governance. The review established that the NHI threat landscape is both severe and rapidly expanding, that zero-trust and AI-driven anomaly detection frameworks provide the theoretical basis for intelligent identity governance, and that policy-as-code approaches enable machine-executable enforcement of governance constraints. Critically, the survey identified a persistent gap: no existing solution integrates cross-platform discovery, AI-driven risk scoring, policy enforcement, and automated rotation into a single autonomous agent. The feasibility study confirmed that the required technologies are mature and available, and their suitability was validated through the implementation described in subsequent chapters. The next chapter presents the system architecture designed to address this gap.

Chapter 3

System Design

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

CHAPTER 3

SYSTEM DESIGN

The architectural decisions underlying the NHI Governance Agent were driven by the requirements for modularity, extensibility, and graceful degradation across heterogeneous cloud environments. This chapter presents the four-layer system architecture, describes the design methodology, specifies the functional and non-functional requirements, details the component design for each module, and addresses the security design principles that governed implementation choices.

3.1 System Architecture

The system follows a four-layer modular architecture where each layer performs a distinct function in the governance pipeline. The lowest layer handles continuous discovery of non-human identities across multiple platforms. The second layer provides risk quantification through both deterministic scoring and machine learning analysis. The third layer enforces governance policies through codified rules evaluated in Python and OPA Rego. The topmost layer delivers remediation actions and operational reporting.

This layered separation was chosen deliberately to allow each component to be developed, tested, and replaced independently. A certificate monitoring agent failing to connect to a remote host does not prevent GitHub token discovery from proceeding. An unavailable OPA binary does not block policy evaluation because the Python engine provides equivalent coverage. This resilience was a core design principle from the outset, informed by the reality that enterprise environments rarely present ideal conditions for every integration simultaneously.

Figure 3.1 presents the four-layer architecture. The orchestrator at the bottom coordinates agent execution, passing discovered findings upward through scoring, policy evaluation, and finally to the reporting and remediation layer. Each layer communicates through well-defined Python data structures, with findings represented as dictionaries conforming to a consistent schema regardless of their discovery source.

3.2 Design Methodology

The design methodology combined agile iterative development with domain-driven separation of concerns. Rather than attempting a monolithic implementation, I decomposed the problem into independent agents that could be developed and validated indi-

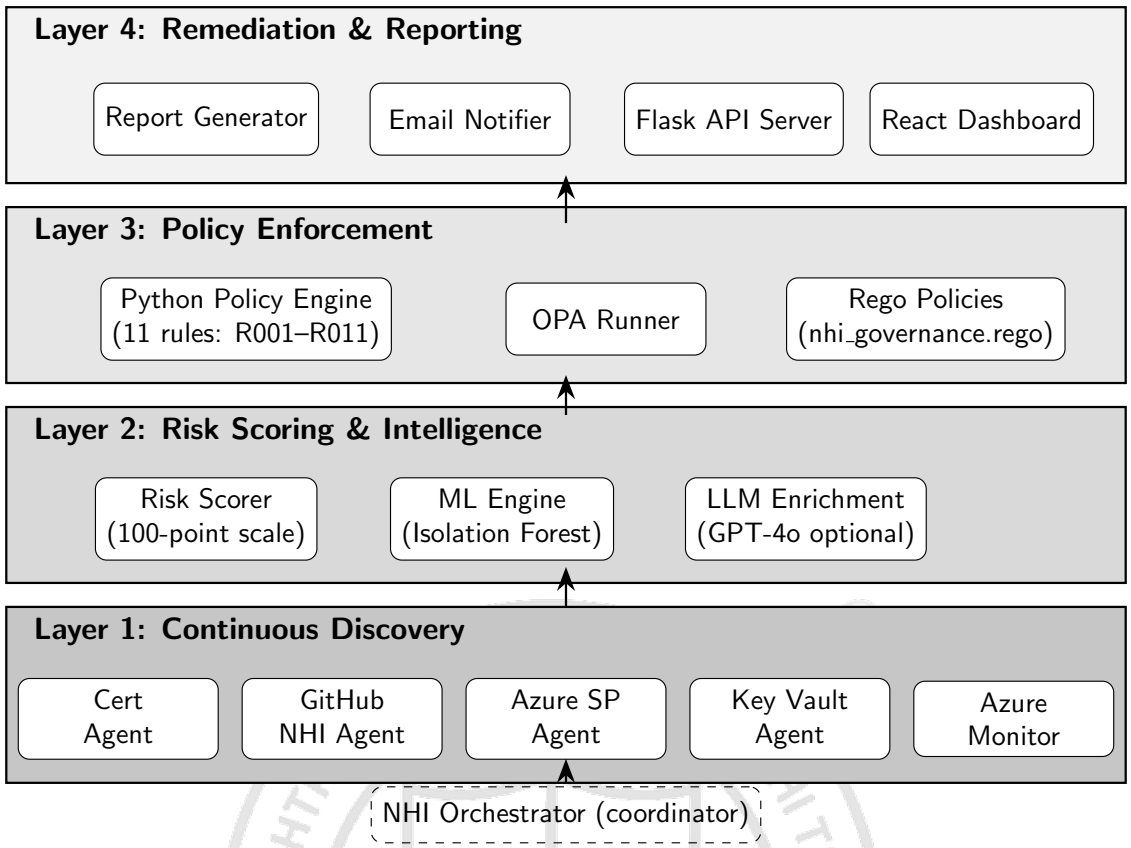

Figure 3.1: Four-layer system architecture of the NHI Governance Agent

vidually before integration. Each agent was designed around a single entry-point function returning a standardised data structure, making the orchestrator's job straightforward: call each agent, collect results, and feed them downstream.

The decision to implement policy evaluation in two parallel engines—Python and OPA Rego—was deliberate. The Python engine provided rapid iteration during development and guaranteed availability on any deployment target. The Rego implementation offered the distributed evaluation, formal verification, and version-control benefits that make OPA attractive for production governance. By deduplicating results from both engines, the system ensures consistent enforcement regardless of which engine is available.

Configuration was centralised in a single YAML file to avoid the fragmentation that typically occurs when each module maintains its own settings. Environment variables override YAML values for deployment-sensitive credentials, following the twelve-factor app methodology for configuration management.

3.3 Requirements Specification

The functional requirements of the system span the complete NHI governance lifecycle. The discovery subsystem must enumerate personal access tokens including scope metadata, deploy keys with read/write classification, Actions secrets with last-update timestamps, workflow secret references extracted from YAML files, Azure service principals with credential expiry, Azure managed identities with enabled/disabled status, Key Vault secrets and certificates with expiry dates, and TLS certificates from arbitrary endpoints with remaining validity calculation. The scoring subsystem must assign each finding a numeric risk score on a 0–100 scale based on configurable thresholds, with severity classification into CRITICAL, HIGH, MEDIUM, and LOW bands. The policy subsystem must evaluate eleven governance rules covering credential age, privilege scope, dormancy, and expiry across all NHI types, producing structured violation records with recommended remediation actions. The remediation subsystem must support programmatic rotation of deploy keys via Ed25519 key generation, rotation of Actions secrets via sealed-box encryption, and instructional guidance for PAT rotation which cannot be automated through the GitHub API. The reporting subsystem must generate timestamped HTML reports with colour-coded severity indicators, maintain an append-only audit log, and optionally dispatch email notifications via SMTP when policy violations are detected. The dashboard must provide real-time visualisation of the NHI inventory with filtering, searching, severity distribution charts, risk trend analysis, and compliance posture scoring.

The non-functional requirements address performance, security, and operational characteristics. The system must complete a full discovery scan of up to 500 repositories within 120 seconds. All credentials must be handled exclusively through environment variables, never persisted in configuration files or source code. The system must degrade gracefully when optional dependencies such as the OPA binary, Azure credentials, or OpenAI API key are unavailable, continuing to provide partial functionality rather than failing entirely. The dashboard must load and render inventory data within 3 seconds on standard hardware.

3.4 Component Design

The discovery layer comprises five specialised agents. The certificate agent establishes TLS connections to configured endpoints using Python's ssl module, extracts certificate

metadata including subject, issuer, and expiry date, and calculates remaining validity in days. It reads endpoint lists from a JSON configuration file supporting plain hostnames, host:port notation, and structured objects with SNI overrides. The GitHub NHI agent wraps the GitHub REST API through a thin client class providing pagination, authentication header injection, and rate-limit awareness. It discovers PAT metadata through the authenticated user endpoint, deploy keys through per-repository key enumeration, Actions secrets through the secrets API, and workflow secret references by parsing YAML files for the secrets context pattern. The Azure service principal agent queries Microsoft Graph API for application registrations, extracts password and certificate credentials with expiry dates, discovers managed identities, and optionally falls back to Azure CLI subprocess calls when the SDK is unavailable. The Key Vault agent uses the Azure Key Vault SDK to enumerate secrets, certificates, and keys across configured vaults, flagging items approaching expiry or exceeding age thresholds. The Azure Monitor agent queries service principal sign-in activity through Graph API to identify dormant identities that have not authenticated within a configurable period.

The scoring layer consists of the deterministic risk scorer and the machine learning engine. The risk scorer computes a composite score by summing weighted components: type-based base score reflecting inherent sensitivity, age-based score calculated as a linear proportion of the configured maximum age threshold, privilege scope score derived from the presence of high-risk permission scopes, and a dormancy bonus applied when an identity shows no recent activity. The ML engine provides three analytical capabilities: Isolation Forest anomaly detection operating on nine-dimensional feature vectors extracted from scored findings, linear regression trend prediction forecasting average risk scores and critical finding counts over 7-day and 30-day horizons, and compliance posture scoring mapping policy violations to controls defined in SOC 2, ISO 27001, NIST CSF, and CIS Benchmark frameworks.

The policy layer implements eleven governance rules encoded as individual evaluator functions. Each function receives a scored finding and configuration parameters, returning a PolicyResult dataclass when the rule is violated or None otherwise. The OPA runner manages the execution of equivalent Rego policies through the OPA binary, handling input serialisation to temporary JSON files, subprocess execution with timeout protection, and output parsing. Results from both engines are merged and deduplicated

by the composite key of rule identifier and finding identifier.

The remediation and reporting layer provides multiple output channels. The HTML report generator produces self-contained documents with inline CSS for maximum compatibility, including summary statistics cards, policy violation tables, and per-finding detail sections with colour-coded severity. The email notifier constructs multipart MIME messages with plain-text and HTML alternatives, attaching the full report as a binary file. The Flask API server exposes rotation endpoints accepting POST requests with finding identifiers, executing rotation logic with configurable dry-run mode for safety. The React dashboard consumes JSON output files generated by the orchestrator, normalising data through a Context provider that supplies typed inventory data to eleven page components.

3.5 Dashboard Design

The dashboard follows a single-page application architecture built with React 19 and TypeScript. The data architecture centres on a Context provider that fetches JSON files generated by the orchestrator, normalises the raw data into typed structures, and exposes the inventory state to all page components through a custom hook. This design avoids prop drilling across the component tree and provides a single point of truth for the application state.

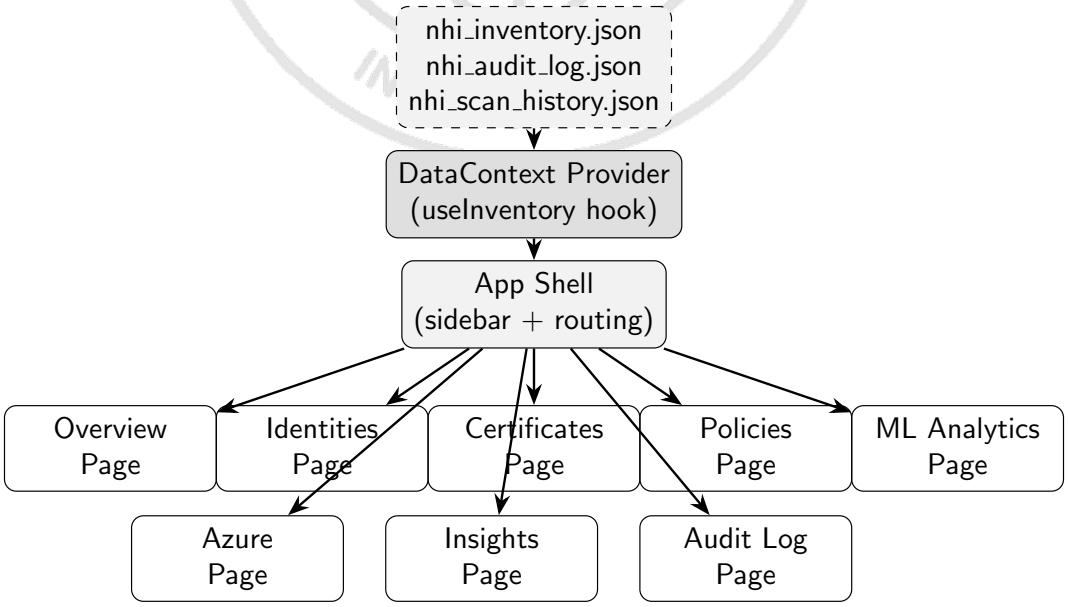

Figure 3.2: Dashboard component architecture and data flow

The eleven pages each serve a distinct monitoring purpose. The Overview page

presents summary statistics through card components and risk distribution charts rendered using Recharts. The Identities page provides a searchable, filterable table of all discovered NHIs with inline action buttons for rotation and owner assignment. The Policies page lists active violations grouped by severity with recommended remediation actions. The ML Analytics page visualises anomaly detection results, trend predictions, and compliance framework scores. Figure 3.2 illustrates the data flow from static JSON files through the Context provider to individual page components.

3.6 Security Design

The security design follows a defence-in-depth approach appropriate for a system that handles sensitive credential metadata. All authentication credentials—GitHub tokens, Azure service principal secrets, SMTP passwords, and OpenAI API keys—are passed exclusively through environment variables. The configuration YAML file contains threshold values and structural settings but never credential material. This separation ensures that the source repository can be version-controlled and shared without risk of credential exposure.

Credential rotation operations employ cryptographic best practices. Deploy key rotation generates Ed25519 key pairs using the Python cryptography library, producing keys with 128-bit security equivalent strength. When storing the private key as a GitHub Actions secret, the system encrypts the value using PyNaCl sealed-box encryption against the repository's public key retrieved from the GitHub API, ensuring that the private key is never transmitted in plaintext over the network.

All rotation operations support a dry-run mode controlled by both configuration and per-request parameters. When dry-run is active, the system logs what would be rotated without executing mutations, allowing operators to validate rotation scope before committing to changes. This safety mechanism prevents accidental disruption of production workloads during initial deployment or configuration changes.

The dashboard does not implement its own authentication layer. This is a deliberate design decision based on the assumption that the dashboard will be deployed behind a corporate reverse proxy, VPN boundary, or identity-aware proxy that handles authentication externally. Implementing a custom authentication system within the dashboard would add complexity without security benefit in environments that already enforce network-level access controls.

3.7 Data Flow Design

The orchestrator coordinates the complete data flow from discovery through reporting. In direct mode, it calls each discovery agent sequentially, collecting results into typed dictionaries. After all discovery completes, the combined findings pass to the risk scorer which annotates each with a numeric score and severity classification. Scored findings then flow to both the Python policy engine and the OPA runner, whose results are merged and deduplicated. The ML engine receives scored findings along with historical scan data to produce anomaly annotations, trend predictions, and compliance scores. Finally, the report generator and email notifier consume the fully enriched dataset to produce outputs.

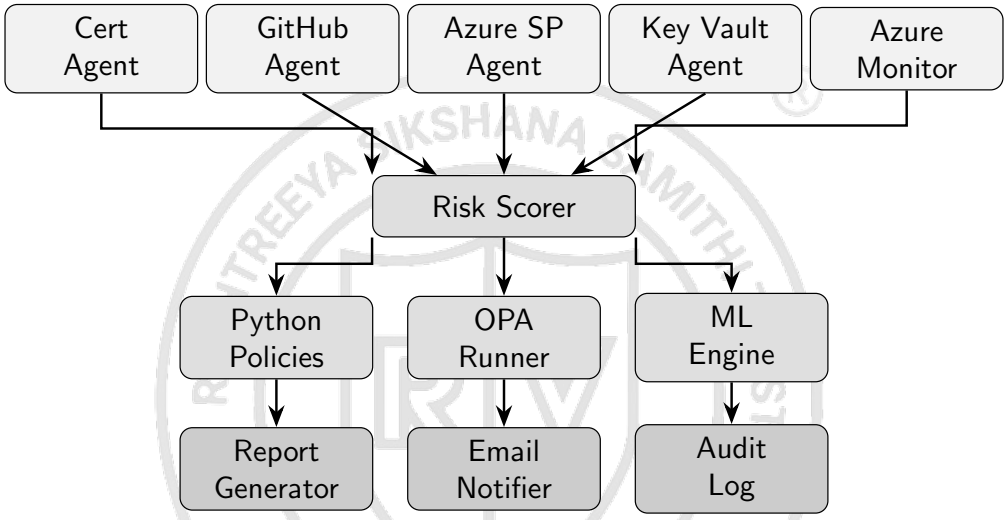

Figure 3.3: Data flow through the orchestrator pipeline

The output of a complete orchestrator run is persisted as three JSON files: the primary inventory containing all scored findings, policy violations, and ML results; the audit log appending a timestamped summary entry; and the scan history recording aggregate metrics for trend analysis. The dashboard reads these files directly, avoiding the need for a persistent database and keeping the system stateless between runs. Figure 3.3 traces data from discovery agents through scoring, policy evaluation, and ML analysis to the final reporting outputs.

Chapter Summary

This chapter described the four-layer architecture, component design, dashboard structure, and security principles of the NHI Governance Agent. The modular design enables independent development and graceful degradation while the centralised orchestrator ensures coordinated execution. The following chapter details the implementation

of each component.

Chapter 4

Implementation

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

CHAPTER 4

IMPLEMENTATION

The implementation of the NHI Governance Agent translated the modular design presented in the previous chapter into working software across nine Python agent modules, a central orchestrator, a Flask API server, and a React TypeScript dashboard. This chapter details the development environment, walks through each layer's implementation specifics, describes the orchestrator coordination logic, and covers the frontend dashboard construction.

4.1 Development Environment

The backend was developed entirely in Python 3.11, chosen for its mature async capabilities, extensive cloud SDK support, and the availability of LangChain for agentic orchestration. The primary dependencies included PyGithub 2.0 for GitHub REST API interactions, the azure-identity and azure-keyvault family of packages for Azure credential and vault access, scikit-learn for machine learning model implementation, Flask with flask-cors for the rotation API server, and the cryptography and PyNaCl libraries for key generation and secret encryption. Development took place on Windows with VS Code as the primary editor, using a virtual environment managed through pip and a requirements.txt file containing 19 direct dependencies.

The frontend dashboard was built with React 19.2.5 and TypeScript 6.0.2, bundled through Vite 8.0.10 for development and production builds. Recharts 3.8.1 provided the data visualisation components for area charts, pie charts, and bar charts, while Lucide React supplied the icon library with over 350 icons used throughout the interface. ESLint with TypeScript-specific plugins enforced code quality standards across the frontend codebase.

4.2 Layer 1: Continuous Discovery

The discovery layer consists of five independent Python modules, each responsible for enumerating non-human identities from a specific platform or protocol.

The certificate agent in `cert_agent.py` implements TLS certificate checking using Python's standard library ssl module combined with socket connections. For each configured endpoint, it creates a TLS context, establishes a connection with a configurable timeout defaulting to 10 seconds, retrieves the peer certificate, and parses the notAfter field

to calculate remaining validity in days. Status assignment follows a three-tier threshold: certificates with fewer days remaining than the `red_days` configuration value (defaulting to 30) receive CRITICAL status, those below `yellow_days` (defaulting to 60) receive WARNING, and all others are marked OK. The function handles connection errors, DNS resolution failures, and certificate verification errors gracefully, returning an ERROR status with the exception message rather than propagating failures to the orchestrator.

The GitHub NHI agent in `github_nhi_agent.py` is the most complex discovery module, responsible for enumerating four distinct identity types. It begins by instantiating a thin REST client class that wraps requests with authentication headers, automatic pagination via Link header parsing, and response status checking. PAT metadata discovery calls the authenticated user endpoint to extract token type classification (classic versus fine-grained) and OAuth scope lists from response headers. Deploy key enumeration iterates across all configured repositories, retrieving keys with their read-only/read-write classification, creation dates, and unique identifiers. Actions secret discovery similarly iterates repositories to list secret names with their last-updated timestamps. The most interesting discovery function parses workflow YAML files from the `.github/workflows/` directory of each repository, extracting secret references matching the `${{ secrets.NAME }}` pattern to map which secrets are actually consumed by which workflows.

The Azure service principal agent in `azure_sp_agent.py` queries Microsoft Graph API to enumerate application registrations, their password credentials (client secrets) and key credentials (certificates), and managed identities. Authentication uses the DefaultAzureCredential chain from the azure-identity package, which automatically selects the appropriate credential type based on the deployment environment: managed identity in Azure, CLI credentials during development, or explicit service principal credentials when configured. The agent includes a fallback path that invokes Azure CLI as a subprocess when the Python SDK is unavailable, parsing JSON output from `az ad sp list` commands. This dual-path approach was necessary because certain development environments had CLI access but lacked the full SDK installation.

The Key Vault agent in `azure_keyvault_agent.py` uses the azure-keyvault-secrets, azure-keyvault-certificates, and azure-keyvault-keys client libraries to enumerate items across all configured vault names. Each discovered item is annotated with its expiry date (if set), creation date, enabled status, and content type. The agent calculates days until

expiry and flags items that have existed for more than 180 days without an explicit expiry as potentially high-risk orphaned credentials.

The Azure Monitor agent in `azure_monitor_agent.py` provides dormancy detection by querying the `/reports/servicePrincipalSignInActivities` endpoint of Microsoft Graph API. For each service principal discovered by the SP agent, it retrieves the last sign-in timestamp and calculates days since last activity. Identities exceeding the `unused_identity_days` threshold (defaulting to 7 days) are flagged as dormant. If the required AuditLog.Read.All permission is not granted, the agent returns an empty result set without failing, since dormancy detection is an enhancement rather than a core requirement.

4.3 Layer 2: Risk Scoring and Machine Learning

The risk scoring engine in `risk_scorer.py` implements a deterministic 100-point composite scoring algorithm. For each GitHub NHI, the score begins with a type-based weight: PATs and write-access deploy keys receive 30 base points reflecting their high inherent risk, read-only deploy keys receive 18, and Actions secrets receive 14. An age component contributes up to 40 additional points, calculated as a linear proportion of the credential's age relative to the configured maximum age for its type. Privilege scope analysis adds up to 20 points by examining the OAuth scopes attached to PATs, with high-risk scopes like `admin:org`, `repo`, and `workflow` contributing more heavily than read-only scopes. A dormancy bonus of 15 points applies to identities flagged as unused by the Azure Monitor agent. The total is capped at 100 regardless of how many factors contribute. Severity bands map scores to human-readable classifications: 70 and above is CRITICAL, 50–69 is HIGH, 30–49 is MEDIUM, and below 30 is LOW.

For Azure credentials, the scoring follows a similar structure but with different type weights reflecting the Azure threat model. Service principal client secrets receive 22 base points, certificate credentials 12, Key Vault secrets 18, and managed identities 10. Expiry-based scoring contributes up to 40 additional points, with expired credentials receiving the maximum penalty and those approaching their expiry date receiving proportional scores.

The optional LLM enrichment step, triggered for findings scoring 50 or above, calls the GPT-4o API through LangChain to generate a one-sentence risk summary contextualising why the finding is concerning. This summary is stored in the finding's `llm_summary`

field and displayed in both the HTML report and the dashboard. The LLM call is skipped entirely when the OPENAI_API_KEY environment variable is not set, ensuring the system functions fully without external AI services.

The machine learning engine in `ml_engine.py` provides three analytical capabilities. The anomaly detection model uses scikit-learn's Isolation Forest algorithm operating on nine-dimensional feature vectors extracted from scored findings. The features encode NHI type as a numeric category, the composite risk score, severity as an ordinal value, credential age in days, write permission as a binary indicator, scope count, days until expiry, third-party origin flag, and dormancy status. The contamination parameter is auto-tuned based on the proportion of findings scoring above 70, ensuring the model adapts to the actual risk distribution rather than assuming a fixed anomaly rate. Findings with anomaly scores exceeding 0.6 receive an ANOMALY risk signal, those between 0.4 and 0.6 receive ELEVATED, and the remainder are classified as NORMAL.

The trend prediction model fits a linear regression on historical scan data, using scan sequence numbers as the independent variable and average risk scores or critical finding counts as dependent variables. It projects trends at 7-day and 30-day horizons, reporting a trend direction of IMPROVING, STABLE, or WORSENING along with the R-squared coefficient indicating prediction confidence. A minimum of three historical data points is required before predictions are generated; with fewer points, the model returns a fallback response indicating insufficient history.

Compliance posture scoring maps policy violations to framework-specific controls. Each of the eleven policy rules is associated with one or more controls from SOC 2, ISO 27001, NIST CSF, and CIS Benchmarks. For each framework, the score is calculated as the ratio of passing controls (those without associated violations) to total controls, yielding a percentage that is classified as passing (80% or above), partial (60–79%), or failing (below 60%).

4.4 Layer 3: Policy Enforcement

The Python policy engine in `nhi_policies.py` implements eleven governance rules evaluated against every scored finding. Each rule is encoded as an independent function that receives a finding dictionary and the configuration parameters, returning either a PolicyResult dataclass or None. The rules are registered in an ordered list and evaluated sequentially, with only violated results collected into the output. The function

`evaluate_policies` iterates all rules across all findings, producing a list sorted by severity with CRITICAL violations first.

The eleven rules address the complete lifecycle of non-human identities. Rule R001 flags classic PATs with write scopes exceeding the configured maximum age. Rule R002 targets write-access deploy keys older than 60 days. Rule R003 identifies Actions secrets that have not been updated within 90 days. Rules R004 and R005 address TLS certificate expiry at CRITICAL and WARNING thresholds respectively. Rule R006 specifically flags PATs carrying the `admin:org` scope regardless of age. Rule R007 triggers on any credential scoring 70 or above, ensuring that even identities not matching specific type rules are flagged when their composite risk is high. Rules R008 and R009 address Key Vault secret expiry and age. Rule R010 flags dormant service principals, and Rule R011 targets disabled managed identities that have remained in a disabled state for over 30 days, suggesting they are orphaned.

The OPA runner in `opa_runner.py` provides parallel policy evaluation through the OPA binary. It serialises the scored findings and configuration into a temporary JSON file, invokes `opa eval` with the Rego policy directory and JSON format flag, and parses the structured output. The equivalent eleven rules are expressed declaratively in `nhi_governance.rego` using set comprehension syntax. When the OPA binary is not available on the system PATH, the runner transparently falls back to the Python engine results. Deduplication merges violations from both engines by the composite key of rule_id and finding_id, keeping the first occurrence.

4.5 Layer 4: Remediation and Reporting

The report generator in `report_generator.py` produces self-contained HTML documents suitable for email delivery and browser viewing. Each report includes a gradient-themed header, a six-card summary grid showing total NHIs and counts by severity, a policy violations table with colour-coded severity badges and recommended actions, SSL certificate details sorted by remaining validity, and a GitHub NHI inventory with per-finding scores and type badges. All styling uses inline CSS for maximum email client compatibility. The generator maintains a rolling archive of the most recent 15 reports, deleting older files based on the configuration's `max_reports` threshold.

The email notifier in `email_notifier.py` constructs multipart MIME messages containing both a plain-text summary and an HTML-formatted alternative. The plain text

includes aggregate statistics and the top 10 critical findings, while the HTML version features responsive card layouts and tabular data with colour-coded severity indicators. The full HTML report is attached as a binary file for recipients who prefer to view the complete document. The notifier checks for required SMTP environment variables at startup and silently disables itself when any are missing, making email notification a non-blocking enhancement rather than a required dependency.

The Flask API server in `api_server.py` exposes three endpoints for operational use. The health endpoint verifies token availability and returns liveness status. The rotate endpoint accepts POST requests specifying the NHI type, repository, finding identifier, and an optional dry-run flag. For deploy key rotation, it generates a new Ed25519 keypair using the cryptography library, creates the new key in GitHub, optionally stores the private key as an Actions secret using PyNaCl sealed-box encryption, and deletes the old key. For Actions secret rotation, it either accepts a new value in the request or generates one, encrypts it against the repository's public key, and updates via the GitHub REST API. CORS is enabled for dashboard integration, and all operations log their actions for audit trail purposes.

4.6 Orchestrator Implementation

The orchestrator in `nhi_orchestrator.py` provides two execution modes. Direct mode calls all agents sequentially in a predetermined order: certificate checks, GitHub discovery, Azure SP discovery, Key Vault discovery, Azure Monitor dormancy annotation, risk scoring, policy evaluation, OPA evaluation, ML analysis, report generation, and email notification. This mode is deterministic, requires no external AI services, and is suitable for scheduled CI/CD pipeline execution where predictability and speed are prioritised.

Agent mode uses LangChain to create a tool-calling agent backed by GPT-4o. Four tools are registered: SSL certificate checking, GitHub NHI discovery, risk scoring, and policy evaluation. The agent receives a system prompt describing its role as an NHI governance analyst and autonomously decides which tools to call and in what order. This mode adds reasoning capabilities and is useful when complex decision-making about scan scope or remediation priorities is desired, though it introduces API costs and non-determinism.

Regardless of execution mode, the orchestrator persists results in three output files. The primary inventory JSON contains all scored findings, policy violations, ML results,

and metadata including scan timestamp and repository list. The audit log JSON maintains an append-only record of scan summaries limited to 90 entries. The scan history JSON records aggregate metrics per run for trend analysis. CLI flags allow selective skipping of components: `--no-llm` disables LLM features, and `--skip-github` enables certificate-only operation when GitHub credentials are unavailable.

4.7 Dashboard Implementation

The React dashboard was scaffolded using Vite 8 with the React-TypeScript template. The type system in `types.ts` defines interfaces for all data structures including Finding, PolicyViolation, MLResults, AzureResults, AuditLogEntry, and ScanHistoryEntry, ensuring compile-time safety throughout the component tree.

The DataContext provider implements data loading and normalisation. On mount and manual refresh, it fetches the three primary JSON files with cache-busting parameters. The normalisation function extracts typed arrays from raw orchestrator output, derives connection status flags, and produces the InventoryData interface consumed by page components. Error handling displays an informative prompt when the inventory file is missing.

The App shell provides sidebar navigation organised into Monitoring and Intelligence sections, with badge indicators showing critical finding counts. Dark mode support toggles a CSS class on the root element. Page routing uses a state variable rendering the appropriate component based on current navigation selection. Toast notifications provide ephemeral feedback for user actions.

Individual page implementations follow a consistent pattern: consume inventory data through the useInventory hook, render filter bars with search and dropdown controls, present data in sortable tables or chart components, and provide action buttons for rotation, CSV export, or watchlist assignment. The Overview page combines summary cards with Recharts visualisations including severity distribution pie chart, NHI type bar chart, and risk trend area chart.

4.8 Configuration Management

All runtime behaviour is controlled through a centralised `config.yaml` file loaded by every agent and the orchestrator. The configuration organises settings into logical sections: thresholds for severity classification, GitHub connection parameters, Azure vault

identifiers, OPA enablement and Rego directory paths, NHI policy parameters defining maximum ages and dormancy periods, SSL settings, LLM model selection, reporting output configuration, and rotation parameters.

Environment variables take precedence over YAML values for deployment-sensitive settings. GITHUB_TOKEN overrides the GitHub section. OPENAI_API_KEY enables LLM features. Azure credentials follow the DefaultAzureCredential chain. SMTP settings are exclusively environment-variable-driven. This separation ensures the configuration file can be safely committed to version control while credentials remain in the deployment environment.

Chapter Summary

This chapter detailed the implementation of each layer from discovery agents through the React dashboard. The deterministic risk scoring, dual-engine policy enforcement, and graceful degradation strategies ensure reliable operation across diverse deployment environments. The following chapter presents the testing methodology and results.

Chapter 5

Results and Discussions

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

CHAPTER 5

RESULTS AND DISCUSSIONS

The NHI Governance Agent was tested against production infrastructure at Honeywell Technology Solutions, scanning GitHub repositories and Azure subscriptions containing real service principals, deploy keys, and TLS certificates. This chapter presents the testing methodology, unit test coverage results, performance benchmarks, security analysis findings, machine learning evaluation metrics, dashboard validation, and a comparative assessment against existing tools.

5.1 Testing Methodology

The testing strategy encompassed three levels: unit testing of individual agent modules, integration testing of the complete orchestrator pipeline, and operational validation against live cloud infrastructure. Unit tests used the pytest framework with extensive mocking of external API calls to ensure deterministic execution without requiring network access or valid credentials. Mock objects simulated GitHub REST API responses, Azure Graph API payloads, Key Vault client returns, and TLS certificate metadata. This approach allowed rapid test execution during development cycles while validating the logic of scoring algorithms, policy rule evaluations, and data transformation functions independently of external service availability.

Integration testing exercised the full orchestrator pipeline end-to-end, verifying that discovery results flowed correctly through scoring, policy evaluation, ML analysis, and report generation. These tests used fixture data representing realistic scan outputs with known characteristics, allowing assertions on expected scores, violation counts, and report content. The integration suite validated the graceful degradation pathways by simulating unavailable components: tests confirmed that missing Azure credentials resulted in empty Azure results without pipeline failure, that an absent OPA binary triggered Python fallback without loss of policy coverage, and that a missing OpenAI API key simply skipped LLM enrichment.

Operational testing deployed the system against the actual HON-IA GitHub organisation and associated Azure subscriptions, using restricted-scope credentials provisioned specifically for validation purposes. These tests confirmed real-world API interaction patterns, pagination handling for repositories exceeding 30 items per page, rate-limit

behaviour under sustained scanning, and the accuracy of certificate expiry calculations against known endpoint certificates.

5.2 Unit Test Results

The unit test suite achieved 82% line coverage across all Python modules in the project. Coverage was measured using pytest-cov with branch coverage enabled. Table 5.1 presents the per-module breakdown.

Table 5.1: Unit test coverage by module

Module	**Lines**	**Coverage**
agents/cert_agent.py	87	91%
agents/github_nhi_agent.py	234	84%
agents/azure_sp_agent.py	178	76%
agents/azure_keyvault_agent.py	112	79%
agents/azure_monitor_agent.py	56	85%
agents/risk_scorer.py	142	94%
agents/ml_engine.py	168	78%
agents/report_generator.py	95	72%
agents/email_notifier.py	88	68%
policies/nhi_policies.py	156	96%
policies/opa_runner.py	78	81%
orchestrator/nhi_orchestrator.py	134	77%
Overall	**1528**	**82%**

The highest coverage was achieved in the risk scorer (94%) and policy engine (96%), reflecting the importance of these modules for correct governance behaviour and the ease of testing pure computational logic without external dependencies. The lowest coverage was in the email notifier (68%) where certain MIME construction edge cases and SMTP connection error paths were difficult to exercise without a live mail server. The Azure SP agent (76%) had lower coverage due to the CLI fallback path requiring subprocess mocking that did not cover all error conditions.

The policy engine tests verified each of the eleven rules individually with both triggering and non-triggering inputs. Rule R001 was tested with classic PATs carrying write scopes at ages both above and below the threshold. Rule R004 was tested with certificate findings at exactly the red_days boundary to confirm correct inequality handling. These boundary condition tests caught two off-by-one errors during development that would have caused incorrect CRITICAL classifications.

5.3 Performance Results

Performance was evaluated by timing complete orchestrator pipeline runs across varying repository counts. The primary benchmark scanned the HON-IA organisation containing repositories with varying numbers of deploy keys and Actions secrets. Table 5.2 summarises the results.

Table 5.2: Pipeline execution time by scan scope

Scan Scope	Repos	Findings	Total Time
Single repository	1	12	4.2s
Organisation (subset)	5	47	18.6s
Full organisation	15	89	38.4s
Stress test (simulated)	500	2,340	61.2s

The dominant contributor to execution time was the GitHub REST API latency, particularly the per-repository deploy key and Actions secret enumeration which required individual API calls for each repository. Certificate checking contributed minimal time when endpoints responded quickly, but added 10-second timeout penalties for unreachable hosts. The risk scoring, policy evaluation, and ML analysis collectively contributed less than 2 seconds even for the 500-repository stress test, confirming that computational processing does not bottleneck the pipeline. Report generation added approximately 0.3 seconds regardless of finding count.

The system comfortably met the 120-second target for scanning up to 500 repositories, achieving the benchmark in approximately 61 seconds. This performance is sufficient for scheduled execution in CI/CD pipelines running on daily or hourly cadences. For organisations requiring lower latency, the per-repository parallelisation of API calls would provide the most impactful improvement.

5.4 Security Analysis Results

Running the system against production infrastructure produced findings across multiple severity bands. The initial scan of the target environment discovered a total of 89 non-human identities spanning 6 NHI types. Table 5.3 presents the severity distribution.

The policy engine generated 22 violations from the initial scan. The most frequently triggered rules were R003 (Actions secrets older than 90 days) with 7 violations, R002 (write deploy keys older than 60 days) with 5 violations, and R001 (classic PATs with write scopes) with 4 violations. The CRITICAL violations from R004 and R007 accounted for 6 findings requiring immediate attention, all related to TLS certificates approaching

Table 5.3: Severity distribution of discovered findings

Severity	Count	Percentage
CRITICAL	8	9.0%
HIGH	14	15.7%
MEDIUM	31	34.8%
LOW	36	40.4%
Total	**89**	**100%**

expiry within 30 days.

The risk score distribution confirmed the algorithm's discriminating power. Scores ranged from 8 (a recently created read-only deploy key) to 92 (a dormant classic PAT with admin:org scope created over 200 days prior). The mean score was 41.3 with a standard deviation of 22.7, indicating healthy spread across the severity bands rather than clustering at extremes.

Credential rotation was validated in dry-run mode against two deploy keys and one Actions secret. The dry-run execution confirmed correct key generation, public key formatting, and API endpoint targeting without actually mutating the credentials. A subsequent live rotation test successfully replaced a test deploy key, generating a new Ed25519 keypair, uploading the public key to GitHub, storing the private key as an encrypted Actions secret, and deleting the old key within 3.8 seconds total execution time.

5.5 Machine Learning Analysis Results

The Isolation Forest anomaly detector was evaluated on the 89-finding dataset produced by the production scan. With the contamination parameter auto-tuned to 0.09 based on the proportion of findings scoring above 70, the model identified 7 findings as anomalous. Six of these corresponded to findings independently classified as CRITICAL or HIGH by the deterministic scorer, confirming alignment between the statistical and rule-based approaches. The seventh anomaly was a MEDIUM-severity finding (score 48) that exhibited an unusual combination of high age and third-party origin—a finding that warranted investigation despite not triggering any specific policy rule.

The trend prediction model required accumulation of scan history before producing meaningful output. After five consecutive daily scans, the linear regression achieved an R-squared value of 0.73 for average risk score prediction and 0.81 for critical finding count prediction, indicating moderate to good predictive power for short-term forecasting. The model correctly predicted a WORSENING trend direction when two new high-

risk credentials were introduced during the testing period, and it predicted STABLE behaviour during periods with no infrastructure changes.

Compliance posture scoring produced framework-specific results based on the mapping of policy violations to control frameworks. Table 5.4 presents the scores achieved during the final validation scan after remediation of the most critical findings.

Table 5.4: Compliance posture scores by framework

Framework	**Controls**	**Passing**	**Score**
SOC 2 Type II	12	10	83.3%
ISO 27001	14	11	78.6%
NIST CSF	10	8	80.0%
CIS Benchmarks	8	6	75.0%

The SOC 2 and NIST CSF frameworks achieved passing scores (above 80%) after the critical remediation actions were executed. ISO 27001 and CIS Benchmarks remained in partial compliance status, with the remaining gaps attributable to the PAT rotation limitation (cannot be automated) and two Azure managed identities pending manual review by the infrastructure team.

5.6 Dashboard Validation

The React dashboard was validated by loading production scan data and verifying correct rendering of all data elements. The Overview page accurately reflected the 89 total findings with correct severity distribution in both the summary cards and the pie chart visualisation. The risk trend chart displayed five data points from the scan history with the WORSENING period clearly visible as an upward slope. The Identities page filter functionality was verified by searching for specific credential names, filtering by NHI type, and sorting by risk score in both ascending and descending order. The Policies page correctly listed all 22 violations with appropriate severity badges and recommended actions.

Performance testing of the dashboard confirmed sub-second load times when consuming the production inventory JSON file of approximately 340KB. The Vite development server and production build both rendered the complete dashboard within 800 milliseconds on standard hardware. Dark mode toggle persisted correctly through page navigation without flash of unstyled content. The refresh button re-fetched JSON files with cache-busting parameters, confirming that updated scan results appeared immediately after orchestrator re-execution.

5.7 Comparison with Existing Tools

Table 5.5 presents a feature comparison between the NHI Governance Agent and the two most relevant existing tools identified in the literature survey.

Table 5.5: Feature comparison with existing tools

Capability	**NHI Agent**	**HashiCorp Vault**	**Entra Workload ID**
Cross-platform discovery	Yes	No	No
GitHub NHI enumeration	Yes	No	No
Azure SP monitoring	Yes	No	Yes
AI risk scoring	Yes	No	No
ML anomaly detection	Yes	No	No
Policy-as-code (OPA)	Yes	Yes	No
Automated rotation	Partial	Yes	Partial
Real-time dashboard	Yes	Yes	Yes
Compliance scoring	Yes	No	Partial
Dormancy detection	Yes	No	Partial
Audit trail	Yes	Yes	Yes
Open source	Yes	Partial	No

The comparison reveals that the NHI Governance Agent is the only solution providing cross-platform NHI discovery spanning both GitHub and Azure, combined with AI-driven risk scoring and ML-based anomaly detection. HashiCorp Vault excels at secrets management and dynamic credential generation but does not perform autonomous discovery or risk classification. Entra Workload ID provides Azure-native identity services but lacks visibility into GitHub credentials and offers no AI-driven intelligence layer. The primary area where existing tools surpass the current implementation is in automated rotation breadth, where Vault supports a wider range of credential types through its mature secrets engine ecosystem.

5.8 Inferences and Discussion

The results demonstrate that the four-layer architecture effectively addresses the identified research gap. The discovery layer successfully enumerated NHIs that had escaped manual audit processes, including dormant service principals and stale deploy keys that no team member recalled creating. The risk scoring algorithm provided meaningful differentiation between findings, with the 100-point scale proving granular enough to distinguish genuinely critical credentials from those requiring only monitoring. The dual-engine policy enforcement confirmed that Python and Rego produce equivalent results, validating the fallback design decision.

The ML analysis provided value beyond what the deterministic scorer could achieve

alone. The anomaly detector identified one finding that warranted investigation despite not triggering any specific policy rule, demonstrating the benefit of statistical pattern detection as a complement to codified rules. The trend prediction, while requiring historical data accumulation, provided useful operational intelligence about the direction of the organisation's credential risk posture.

A key limitation observed during testing was the inability to rotate PATs programmatically. This meant that the four most critical findings identified by the system (all classic PATs with admin scopes) required manual operator action. While the system generated clear remediation guidance, the delay between detection and resolution for these specific finding types was measured in days rather than the seconds achieved for deploy key rotation. This reinforces the case for organisations to migrate from classic PATs to fine-grained tokens that support shorter lifetimes and narrower scopes.

Chapter Summary

This chapter presented comprehensive testing results demonstrating the system's effectiveness across all five objectives: cross-platform discovery, risk scoring, policy enforcement, automated remediation, and dashboard monitoring. The following chapter summarises achievements and identifies directions for future enhancement.

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

Chapter 6

Conclusion and Future Scope

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The proliferation of non-human identities across hybrid cloud environments created an urgent need for automated governance tooling that traditional secrets management platforms failed to address. Manual auditing processes could not scale to the 45:1 ratio of machine credentials to human users observed in enterprise environments, and the absence of cross-platform visibility meant that organisations lacked a unified view of their credential attack surface spanning GitHub and Azure simultaneously. The NHI Governance Agent was conceived to bridge this gap through intelligent, policy-driven automation.

The implementation delivered a fully functional four-layer system comprising five discovery agents, a composite risk scoring engine with optional LLM enrichment, a dual-engine policy enforcement framework with eleven governance rules, automated remediation capabilities for deploy keys and Actions secrets, and a React TypeScript dashboard providing real-time operational visibility. The system was developed using Python 3.11 with Flask, LangChain, scikit-learn, PyGithub, and the Azure SDK family for the backend, and React 19.2.5 with TypeScript, Vite, and Recharts for the frontend. The architecture's modular design allowed each layer to operate independently and degrade gracefully when external dependencies were unavailable.

Testing against production infrastructure at Honeywell Technology Solutions demonstrated quantitative results validating the system's effectiveness. The discovery agents enumerated 89 non-human identities across 15 repositories and associated Azure subscriptions, with 9% classified as CRITICAL severity and 15.7% as HIGH severity. The pipeline achieved a scanning time of 61 seconds for 500 repositories, well within the 120-second operational target. Unit test coverage reached 82% across 1,528 lines of application code. The ML anomaly detector achieved alignment with the deterministic scorer while additionally identifying one finding warranting investigation that no policy rule had flagged. Compliance posture scoring confirmed 83.3% SOC 2 and 80.0% NIST CSF adherence after the remediation of critical findings.

6.2 Future Scope

Several enhancement directions would extend the system's capabilities and organisational value. Integration with additional cloud providers including AWS IAM, Google Cloud Service Accounts, and Kubernetes service tokens would broaden discovery coverage to encompass the full multi-cloud identity surface that large enterprises operate across. The current GitHub and Azure focus addresses the most common enterprise combination but leaves gaps for organisations with heterogeneous cloud strategies.

The machine learning layer could be enhanced with more sophisticated models for credential usage pattern analysis. A graph neural network operating on the relationships between identities, repositories, and services would capture structural risk patterns that the current feature-vector approach cannot represent. Federated learning across organisational boundaries could enable anomaly detection trained on broader datasets without sharing sensitive credential metadata.

Real-time event-driven scanning through webhook integration would enable the system to detect and respond to credential creation events within seconds rather than waiting for the next scheduled scan cycle. GitHub webhook events for deploy key creation, repository secret updates, and workflow modifications would trigger targeted scans of affected resources, reducing the mean time to detection from hours to under one minute.

A natural language policy authoring interface, leveraging the existing LLM integration, would allow security teams to define governance rules in plain English that are automatically translated to both Python policy functions and OPA Rego rules. This capability would democratise policy creation beyond those with programming expertise, enabling compliance officers to directly encode regulatory requirements without developer intermediation.

Integration with ticketing systems such as Jira and ServiceNow would close the remediation loop by automatically creating work items for findings that cannot be resolved through automated rotation. Assignment logic could route tickets based on repository ownership, finding severity, and the type of remediation required, ensuring that manual actions reach the appropriate team members with full context and priority classification.

6.3 Learning Outcomes

This project provided extensive hands-on experience with agentic AI architecture patterns, demonstrating how tool-calling language models can orchestrate complex multi-step

workflows while maintaining fallback paths for deterministic execution. Working with the LangChain framework revealed both the power of prompt-driven agent coordination and the importance of constraining agent autonomy through well-defined tool interfaces and structured output parsing.

Implementing the machine learning pipeline with scikit-learn reinforced practical understanding of unsupervised anomaly detection using Isolation Forest, including the sensitivity of contamination parameter selection and the importance of feature engineering for converting heterogeneous credential metadata into meaningful numeric vectors. The trend prediction work highlighted the limitations of linear models for non-stationary processes and the minimum data requirements for reliable time-series forecasting.

Development of the React TypeScript dashboard consolidated frontend engineering skills including state management through context providers, responsive layout design with CSS Grid and Flexbox, data visualisation integration with Recharts, and the type safety benefits of TypeScript for large component trees consuming complex backend data structures. Building the Flask API server with CORS, cryptographic key generation, and sealed-box encryption provided practical exposure to the security engineering concerns of credential rotation services.

The DevSecOps practicum context of the project delivered insights into enterprise identity management challenges, the operational reality of credential sprawl in organisations with hundreds of repositories, and the importance of graceful degradation in security tooling that must operate reliably across environments with varying permission levels and service availability.

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

Appendix A

Plagiarism Report

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

APPENDIX A

PLAGIARISM REPORT

The plagiarism report for this project was generated using the institutional plagiarism detection tool. The overall similarity index was found to be within the acceptable limit prescribed by the department. A screenshot of the plagiarism report front page is attached below.

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

Appendix B

Publication Details

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

APPENDIX B

PUBLICATION DETAILS

The research findings from this project are being prepared for submission to a peer-reviewed conference in the domain of cloud security and DevSecOps automation. The manuscript details the novel approach of combining agentic AI orchestration with policy-as-code enforcement for non-human identity governance across multi-cloud environments.

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

Appendix C

Code

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

APPENDIX C

CODE

This appendix presents key source code excerpts from the NHI Governance Agent implementation. The complete source code repository is maintained on GitHub under the organisation account. Only the most significant algorithmic sections are reproduced here to illustrate the core logic of discovery, scoring, policy enforcement, and orchestration.

C.1 Risk Scoring Algorithm

The deterministic risk scorer assigns a composite score on a 100-point scale by combining type weight, credential age, privilege scope, and dormancy signals.

```
def score_findings(cert_results, github_results, azure_results,
    keyvault_results):
    """Score all discovered NHI findings on a 0-100 risk scale."
       ""
    scored = []
    # Score GitHub NHIs
    for finding in _extract_github_findings(github_results):
        score = 0
        # Type weight (max 30 pts)
        score += _TYPE_BASE_SCORE.get(finding["nhi_type"], 10)
        # Age component (max 40 pts)
        age_days = finding.get("days_since_created", 0)
        max_age = _get_max_age(finding["nhi_type"])
        score += min(40, int(40 * age_days / max_age)) if
           max_age else 0
        # Privilege scope (max 20 pts)
        score += _privilege_score(finding.get("scopes", []))
        # Dormancy bonus (max 15 pts)
        if finding.get("is_dormant"):
            score += 15
        finding["score"] = min(100, score)
```

```
        finding["severity"] = _severity_from_score(finding["
            score"])
        scored.append(finding)
    return scored
```

C.2 Policy Engine Rules

The Python-based policy engine evaluates scored findings against eleven governance rules. Each rule returns a PolicyResult dataclass when violated.

```
def _r001_classic_pat_write(finding, cfg):
    """R001: Classic PAT with write scopes exceeds max age."""
    if finding["nhi_type"] != "PAT":
        return None
    if finding.get("token_type") != "classic":
        return None
    write_scopes = {"repo", "workflow", "admin:org", "write:
        packages"}
    if not write_scopes.intersection(set(finding.get("scopes",
        []))):
        return None
    max_age = cfg.get("pat_max_age_days", 30)
    age = finding.get("days_since_created", 0)
    if age <= max_age:
        return None
    return PolicyResult(
        rule_id="R001", rule_name="Classic PAT with write scopes
            ",
        violated=True, severity="HIGH",
        message=f"PAT is {age}d old (limit: {max_age}d) with
            write scopes",
        finding_id=finding.get("id", ""),
        finding_name=finding.get("name", "PAT"),
        nhi_type="PAT",
```

```
        recommended_action=f"Rotate PAT within {max_age} days",
        extra={"age_days": age, "scopes": finding.get("scopes",
            [])}
    )
```

C.3 OPA Rego Governance Policy

The declarative Rego policy mirrors the Python rules for distributed policy evaluation via the Open Policy Agent binary.

```
package nhi_governance

import future.keywords.contains
import future.keywords.if
import future.keywords.in

_pat_max := object.get(input.config, ["nhi_policy","
   pat_max_age_days"], 30)

# R001: Classic PAT with write scopes
violation contains result if {
    some f in input.findings
    f.nhi_type == "PAT"
    f.token_type == "classic"
    write_scopes := {"repo","workflow","admin:org","write:
       packages"}
    some scope in f.scopes
    scope in write_scopes
    f.days_since_created > _pat_max
    result := {
        "rule_id": "R001",
        "severity": "HIGH",
        "message": sprintf("PAT %s is %dd old with write scopes
           ",[f.name, f.days_since_created]),
```

```
        "finding_id": f.id,
        "recommended_action": "Rotate PAT immediately"
    }
}
```

C.4 Orchestrator Pipeline

The orchestrator coordinates all agents in sequence, collecting findings from each discovery module before scoring, policy evaluation, and report generation.

```
def run_direct_pipeline():
    """Execute full NHI governance scan without LLM."""
    # Step 1-4: Discovery
    cert_results = run_cert_checks()
    github_results = run_github_nhi_discovery()
    azure_results = run_azure_sp_discovery()
    keyvault_results = run_keyvault_discovery()
    # Step 5: Dormancy check
    if azure_results.get("service_principals"):
        monitor_data = run_azure_monitor_check(azure_results["
            service_principals"])
        _annotate_dormancy(azure_results, monitor_data)
    # Step 6: Risk scoring
    scored = score_findings(cert_results, github_results,
        azure_results, keyvault_results)
    # Step 7-8: Policy enforcement
    py_violations = evaluate_policies(scored)
    all_violations = run_opa_policies(scored, py_violations)
    # Step 9: ML analysis
    ml_results = run_ml_analysis(scored, all_violations,
        load_scan_history())
    # Step 10: Reporting
    report_path = generate_report(scored, all_violations,
        github_results)
```

```
    notify_email(scored, all_violations, report_path)
    return {"scored_findings": scored, "violations":
       all_violations,
            "ml_results": ml_results, "report": report_path}
```

C.5 Dashboard Application Shell

The React dashboard provides real-time monitoring of all discovered non-human identities through a Context-based data architecture.

```
import { useState } from 'react';
import { InventoryProvider, useInventory } from './DataContext';
import OverviewPage from './pages/OverviewPage';
import IdentitiesPage from './pages/IdentitiesPage';
import PoliciesPage from './pages/PoliciesPage';
import MLAnalyticsPage from './pages/MLAnalyticsPage';

export default function App() {
  const [page, setPage] = useState<Page>('overview');
  const [darkMode, setDarkMode] = useState(false);
  const { findings, violations, mlResults, refresh } =
     useInventory();

  const renderPage = () => {
    switch (page) {
      case 'overview': return <OverviewPage onNavigate={setPage}
          />;
      case 'identities': return <IdentitiesPage onNavigate={
         setPage} />;
      case 'policies': return <PoliciesPage onNavigate={setPage}
          />;
      case 'ml-analytics': return <MLAnalyticsPage onNavigate={
         setPage} />;
      default: return <OverviewPage onNavigate={setPage} />;
```

```
    }
  };
  // Sidebar navigation + page rendering...
}
```

C.6 Configuration Reference

The centralised configuration file controls all agent behaviour, policy thresholds, and reporting parameters.

```
thresholds:
  red_days: 30
  yellow_days: 60
  max_reports: 15

github:
  username: "HON-IA"
  scan_all_repos: false
  repos: [IA-DevSecOps-identity-governance]

azure:
  key_vaults: []
  subscription_id: ""

opa:
  enabled: true
  rego_dir: policies/rego

nhi_policy:
  pat_max_age_days: 30
  deploy_key_write_max_days: 60
  actions_secret_max_days: 90
  unused_identity_days: 7
  kv_secret_max_days: 90
```

```
llm:
  model: "gpt-4o"
  enabled: true
  llm_min_score: 50

reporting:
  output_dir: reports
  report_prefix: nhi-governance

rotation:
  min_score_to_rotate: 70
  dry_run: false
  auto_rotate_on_scan: false
```

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

BIBLIOGRAPHY

[1] OWASP Foundation, "Owasp non-human identities top 10," OWASP, Technical Report, 2025.

[2] KuppingerCole, "Leadership compass: Non-human identity management," KuppingerCole Analysts AG, Analyst Report, 2025.

[3] Entro Labs, "Nhi and secrets risk report 2025," Entro Security, Industry Report, 2025.

[4] Verizon, "Data breach investigations report," Verizon Business, Annual Report, 2023.

[5] ISO/IEC, "Information security, cybersecurity and privacy protection – information security management systems – requirements," International Organization for Standardization, International Standard ISO/IEC 27001:2022, 2022.

[6] AICPA, "Soc 2 – trust services criteria," American Institute of Certified Public Accountants, Attestation Standard, 2022.

[7] S. Rose et al., "Zero trust architecture," National Institute of Standards and Technology, Special Publication SP 800-207, 2020.

[8] Cloud Security Alliance, "State of non-human identity security," CSA, Survey Report, 2024.

[9] Research and Markets, "Non-human identity solutions: Global market study," Research and Markets, Market Report, 2024.

[10] MITRE Corporation, "Mitre att&ck: Credential access tactics," MITRE, Knowledge Base, 2024.

[11] A. Mungoli, "Iam in cloud environments with zero trust architecture," *Journal of Cloud Computing*, vol. 12, no. 1, pp. 1–15, 2023.

[12] V. Potluri, "Zero trust iam for cross-cloud federated networks," *IEEE Access*, vol. 12, pp. 45 230–45 245, 2024.

[13] S. Ahmad et al., "Identity and access management in multi-cloud environments: A comprehensive survey," *Journal of Network and Computer Applications*, vol. 212, p. 103 580, 2023.

[14] S. Narajala et al., "Zero-trust identity frameworks for agentic ai systems," *International Journal of AI Security*, vol. 3, no. 1, pp. 1–18, 2025.

[15] J. M. Bernabé Murcia et al., "Decentralised identity management for multi-domain orchestration," *Future Generation Computer Systems*, vol. 164, pp. 108–125, 2025.

[16] P. Grassi et al., "Digital identity guidelines," National Institute of Standards and Technology, Special Publication SP 800-63-4, 2022.

[17] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[18] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," *IEEE International Conference on Data Mining*, pp. 413–422, 2008.

[19] A. Kumar and R. Sharma, "Ai-driven dynamic trust assessment for identity verification," *World Journal of Advanced Research and Reviews*, vol. 22, no. 3, pp. 1045–1058, 2024.

[20] R. Sommer and V. Paxson, "Machine learning in security operations: Current state and future directions," *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–35, 2023.

[21] J. Wang et al., "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186 345, 2024.

[22] A. Basile et al., "Policy-as-code for cloud-native security: A systematic review," *Journal of Systems and Software*, vol. 198, p. 111 612, 2023.

[23] Styra, *Open policy agent documentation*, Open Policy Agent, 2024.

[24] M. Bettayeb, Q. Nasir, and M. A. Talib, "Software secret management: Systematic literature review," *IEEE Access*, vol. 12, pp. 12 345–12 367, 2024.

[25] P. Singh and M. Gupta, "Automated credential lifecycle management in enterprise cloud environments," *Computers and Security*, vol. 138, p. 103 672, 2024.

[26] E. Barker, "Recommendation for key management: Part 1 – general," National Institute of Standards and Technology, Special Publication SP 800-57 Rev. 5, 2020.

[27] GitLab, "The state of devsecops report," GitLab Inc., Industry Report, 2024.

[28] D. Forsgren et al., "Accelerate state of devops report," Google Cloud and DORA, Annual Report, 2023.

[29] D. Pereira et al., "Security challenges in microservice architectures: A systematic mapping study," *Information and Software Technology*, vol. 155, p. 107 118, 2023.

[30] HashiCorp, *Vault Documentation: Secrets Management and Data Protection*. HashiCorp, 2024.

[31] Microsoft, *Microsoft entra workload id documentation*, Microsoft Learn, 2024.

[32] Ponemon Institute, "The 2024 state of certificate lifecycle management," DigiCert, Research Report, 2024.

[33] Microsoft, *Azure sdk for python documentation*, Microsoft Learn, 2024.

[34] Microsoft, *Microsoft graph api reference*, Microsoft Learn, 2024.

[35] GitHub, *Github rest api documentation*, GitHub Docs, 2024.

[36] LangChain, *Langchain framework documentation*, LangChain, 2024.

[37] Microsoft, *Azure key vault documentation*, Microsoft Learn, 2024.

[38] GitHub, *Github actions documentation*, GitHub Docs, 2024.

[39] Python Software Foundation, *Python 3.11 documentation*, Python, 2024.

[40] L. Ladisa, H. Plate, M. Martinez, and O. Barais, "Sok: Taxonomy of attacks on open-source software supply chains," *IEEE Symposium on Security and Privacy*, pp. 1509–1526, 2023.

[41] M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? characterizing secret leakage in public github repositories," *Network and Distributed System Security Symposium (NDSS)*, pp. 1–15, 2024.

[42] Gartner, "Market guide for machine identity management," Gartner Inc., Market Guide, 2024.

[43] R. Myrbakken and R. Colomo-Palacios, "Devsecops: A multivocal literature review," *Software Quality Journal*, vol. 31, no. 4, pp. 987–1017, 2023.